

Scalable and Elastic Ticket Service **(AWS Managed Systems)**

Índice

Arquitectura	3
Explicación de la interacción de los componentes	3
Corrección	5
Modelo de consistencia	5
Estrategia de control de concurrencia	5
Evaluación del rendimiento	7
Configuración experimental	7
Definición de la carga de trabajo ($Z(t)$)	7
Metodología de las pruebas de estrés	8
Gráficas de validación	9
Evolución del Backlog de la cola en el tiempo (Queue backlog vs time)	9
Rendimiento vs Trabajadores (Throughput vs workers)	11
Percentiles de latencia (Latency percentiles)	13
Análisis	15
Identificación de cuellos de botella (Bottlenecks identified)	15
Comportamiento del escalado (Scaling behavior explained)	15
Discusión de compensaciones (Trade-offs discussed)	15
Preguntas Conceptuales	17
Compromiso entre Consistencia y Escalabilidad	17
Compromisos entre Tolerancia a Fallos y Rendimiento	18
Uso de la IA	19

Arquitectura

Explicación de la interacción de los componentes

El sistema distribuido de venta de entradas se ha diseñado integrando servicios administrados de AWS y máquinas virtuales, con una arquitectura estructurada para garantizar el procesamiento asíncrono, la escalabilidad dinámica y la tolerancia a fallos. La interacción técnica fluye a través de los siguientes componentes clave:

1. **Generación de Carga (Productor):** Un componente externo inyecta la carga de trabajo elástica, denominada $Z(t)$, simulando distintos escenarios de tráfico (desde fases de baja carga hasta picos repentinos de 100 peticiones por segundo). Este productor publica las intenciones de compra directamente en el middleware de mensajería.
2. **Procesamiento Asíncrono (RabbitMQ):** Alojado en una máquina virtual Amazon EC2, RabbitMQ actúa como la cola principal del sistema. Su inclusión es un requisito obligatorio de la arquitectura para actuar como un amortiguador ante variaciones de tráfico, permitiendo el suavizado de la carga (load smoothing) y garantizando el desacoplamiento total entre los productores de mensajes y los consumidores.
3. **Escalado Dinámico (Autoescalador):** Un controlador específico monitoriza en tiempo real la longitud de la cola (Backlog) de RabbitMQ. Aplicando un modelo matemático basado en los mensajes pendientes, el tiempo de respuesta objetivo y la capacidad configurada por worker, el autoescalador se comunica de forma continua con la API de AWS para ajustar los límites de concurrencia de las funciones Lambda. Esto provee elasticidad al sistema, escalando los recursos hacia arriba o hacia abajo dinámicamente y evitando el sobre-aprovisionamiento.
4. **Workers Sin Estado (AWS Lambda):** La capa de ejecución computacional está compuesta por un clúster de AWS Lambda operando como Stateless Workers. Estas instancias consumen los mensajes de la cola con un límite de prefetch de 10, lo que permite un reparto equitativo del trabajo sin saturar a un único consumidor. Durante su ciclo de vida, cada Lambda extrae el mensaje, verifica la idempotencia de la petición utilizando un identificador único (`request_id`), e inyecta un retraso artificial obligatorio de 100 milisegundos dentro de su lógica para simular el tiempo de respuesta de una pasarela de pagos externa real.
5. **Almacenamiento Persistente (PostgreSQL):** Alojada de forma centralizada en una instancia EC2, esta base de datos relacional actúa como la fuente de almacenamiento persistente del sistema. Las funciones Lambda se conectan a ella para registrar el estado de las peticiones, verificar la disponibilidad del inventario y asentar las transacciones finales de los asientos, centralizando en esta capa la responsabilidad de garantizar que no existan ventas duplicadas.

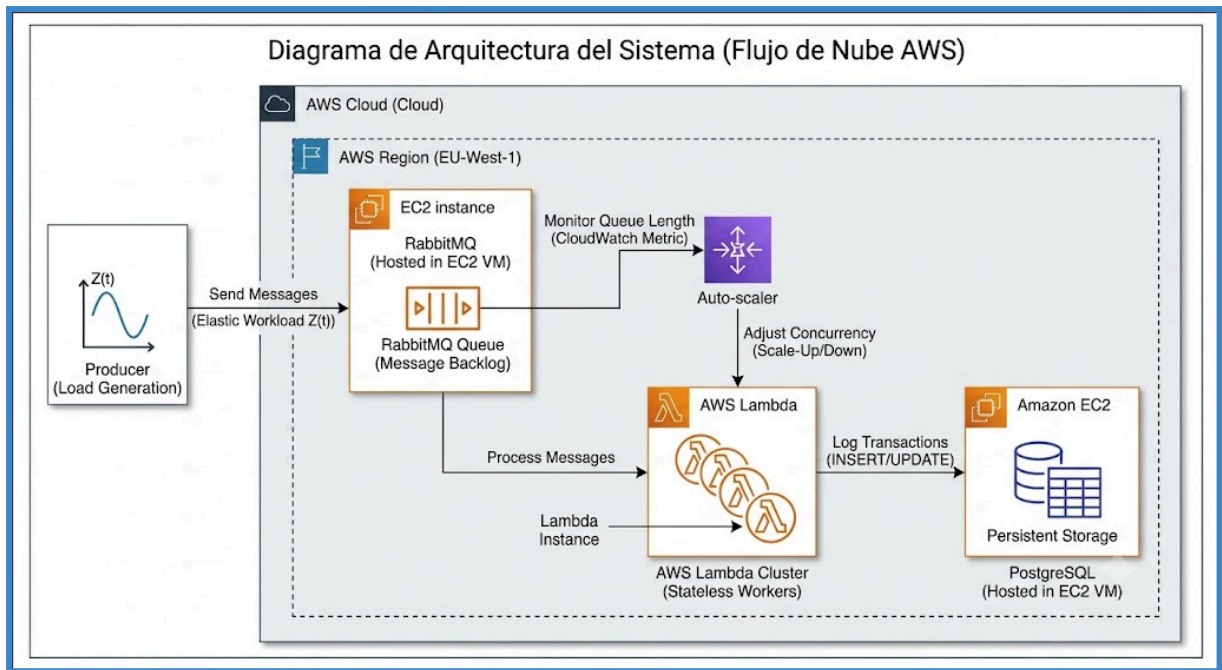


Imagen 1: Diagrama Arquitectura del Sistema

Corrección

Modelo de consistencia

El sistema implementa un modelo de consistencia fuerte (Strong Consistency). Esta decisión arquitectónica responde directamente al requisito funcional estricto que prohíbe de forma absoluta la sobreventa de entradas ("No overselling allowed"), aplicable tanto para el límite de 100.000 pases no numerados como para los asientos numerados individuales. A diferencia de los modelos de consistencia eventual que podrían ser más veloces pero corren el riesgo de presentar anomalías de datos, la consistencia fuerte garantiza que cualquier trabajador verifique y asiente la información sobre el estado real y exacto del inventario, imposibilitando la venta duplicada de un mismo ticket.

Estrategia de control de concurrencia

Para asegurar el modelo de consistencia fuerte y evitar cualquier inconsistencia durante los picos de carga, se ha implementado una estrategia de control de concurrencia mediante bloqueo pesimista (Pessimistic Locking) operando a nivel de fila dentro de la base de datos PostgreSQL. En la capa lógica, cuando un worker (AWS Lambda) procesa una orden, ejecuta la instrucción SQL `SELECT ... FOR UPDATE` de forma previa a cualquier modificación. Este comando adquiere un bloqueo exclusivo sobre la base de datos dependiendo del tipo de entrada:

- **Entradas no numeradas:** Se bloquea la fila central del evento en la tabla `events` (`WHERE event_id = 1`) para descontar el contador general.
- **Entradas numeradas:** Se bloquea únicamente la fila específica del asiento en la tabla `seats` (`WHERE event_id = 1 AND seat_number = %s`).

En escenarios de alta concurrencia donde se producen colisiones cruzadas (como múltiples trabajadores atacando de forma simultánea el mismo asiento VIP en el escenario Hotspot), el motor de PostgreSQL fuerza a las transacciones secundarias a ponerse en cola y esperar a que la transacción principal libere el candado mediante un `COMMIT` o un `ROLLBACK`. Este mecanismo erradica por completo la posibilidad de sufrir condiciones de carrera (race conditions). Aunque esta estrategia introduce demoras físicas en los tiempos de espera transaccionales bajo estrés severo (reflejándose en la latencia del percentil 99), se justifica plenamente como la compensación técnica (trade-off) necesaria para garantizar un 100% de exactitud y robustez en la venta de asientos.

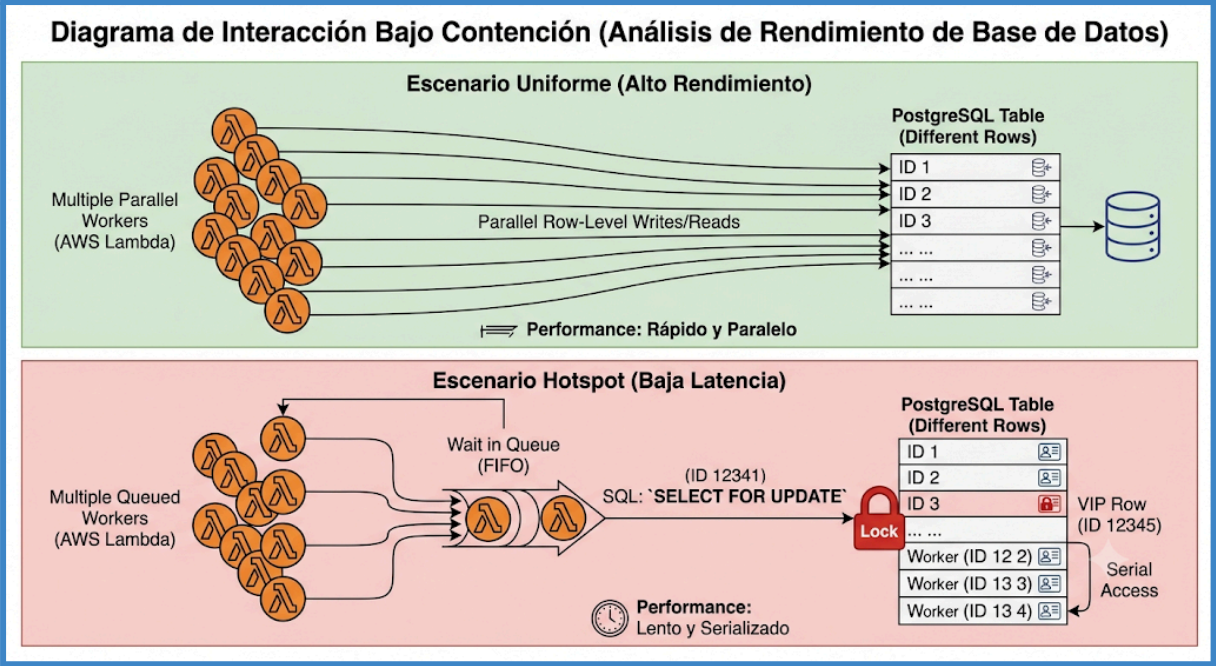


Imagen 2: Diagrama de Interacción Bajo Contención

Evaluación del rendimiento

Configuración experimental

Para asegurar la fiabilidad de las mediciones y simular las condiciones de un entorno de producción realista, el entorno experimental se configuró bajo las siguientes directrices de realismo y control:

- **Latencia Artificial Obligatoria:** Se introdujo un retraso forzado de 100 milisegundos dentro de la lógica de procesamiento de cada trabajador (AWS Lambda) utilizando la instrucción `time.sleep(0.100)`. Este retraso emula la latencia inherente al procesamiento de una pasarela de pagos externa real y es un factor crítico considerado en el cálculo del throughput máximo.
- **Limpieza y Aislamiento de Estado:** Antes de iniciar cada experimento, se ejecutó un script de purga automatizado que vaciaba los mensajes residuales de RabbitMQ (`purge_queue`) y reiniciaba el estado de la base de datos PostgreSQL mediante comandos `TRUNCATE` sobre las tablas transaccionales, restaurando además el inventario base de 100.000 entradas. Esto garantizó que cada prueba arrancara desde un estado completamente limpio.
- **Medición en Servidor (Server-side):** Evitando la inexactitud de las mediciones del lado del cliente, las métricas de latencia de extremo a extremo y el rendimiento (throughput) se calcularon estrictamente en el lado del servidor. Se utilizó la función nativa `clock_timestamp()` de PostgreSQL para registrar con precisión de microsegundos el inicio y la culminación de cada transacción validada y exitosa.

Definición de la carga de trabajo ($Z(t)$)

Para demostrar la capacidad de elasticidad y resiliencia del sistema, se diseñó una carga de trabajo variable en el tiempo, denotada como $Z(t)$. Un generador inyectó mensajes asíncronos en el sistema orquestando cinco fases secuenciales de estrés:

1. **Fase de baja carga (Low load phase):** Inyección de 50 mensajes a una tasa de 5 peticiones por segundo (req/s), para observar el comportamiento en reposo.
2. **Incremento gradual (Gradual ramp-up):** Inyección de 200 mensajes a 20 req/s, evaluando la sensibilidad inicial del escalado.
3. **Picos repentinos (Sudden spikes):** Inyección masiva de 500 mensajes a 100 req/s, forzando la acumulación inmediata en la cola para verificar la respuesta reactiva del autoescalador.
4. **Alta carga sostenida (Sustained high load):** Inyección de 1.500 mensajes a 50 req/s, diseñada para evaluar la estabilidad del procesamiento transaccional continuado.
5. **Fase de enfriamiento (Cool-down phase):** Descenso a 50 mensajes a 5 req/s, con el objetivo de comprobar que el sistema destruye recursos (workers) de forma dinámica para evitar el sobre-aprovisionamiento.

Metodología de las pruebas de estrés

La metodología analítica consistió en incrementar progresivamente la carga transaccional hasta forzar la aparición de inestabilidad, logrando identificar la formación de cuellos de botella temporales, la acumulación en el backlog y el crecimiento de la latencia en el percentil 99.

Para evaluar en profundidad la degradación del rendimiento ocasionada por la contención de bloqueos (Lock contention), estas simulaciones $Z(t)$ se ejecutaron de manera independiente sobre tres escenarios clave:

- **Escenario A (No numeradas):** Las peticiones compiten directamente contra un contador de aforo general en una sola fila transaccional (`event_id = 1`), generando una contención moderada.
- **Escenario B (Carga Uniforme):** Peticiones de asientos numerados distribuidas aleatoriamente entre 100.000 posiciones disponibles. Este enfoque minimiza el choque de transacciones simultáneas y reduce la probabilidad matemática de colisiones de bloqueo en disco casi a cero.
- **Escenario C (Carga Hotspot):** Simulación de un entorno altamente concurrido y desigual donde el 80% de las solicitudes de compra se dirigen deliberadamente al 5% de los asientos ("asientos VIP"). Esta prueba extrema fuerza colisiones severas para revelar la respuesta penalizadora del sistema bajo el control de concurrencia pesimista.

Gráficas de validación

Evolución del Backlog de la cola en el tiempo (Queue backlog vs time)

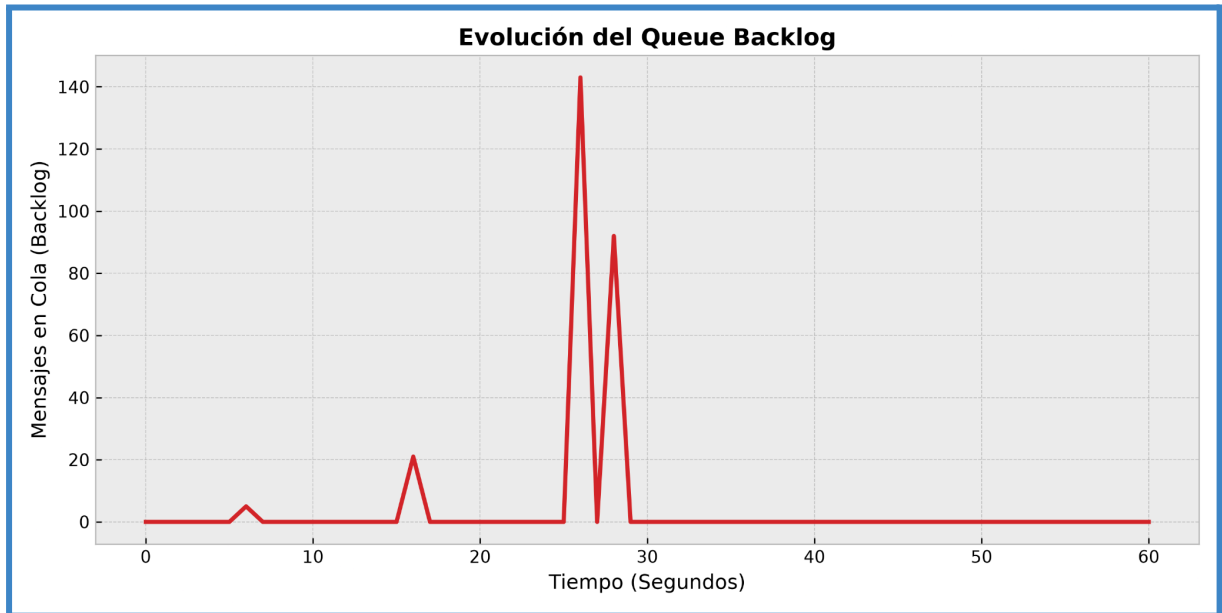


Imagen 3: Backlog en escenario No Numerado

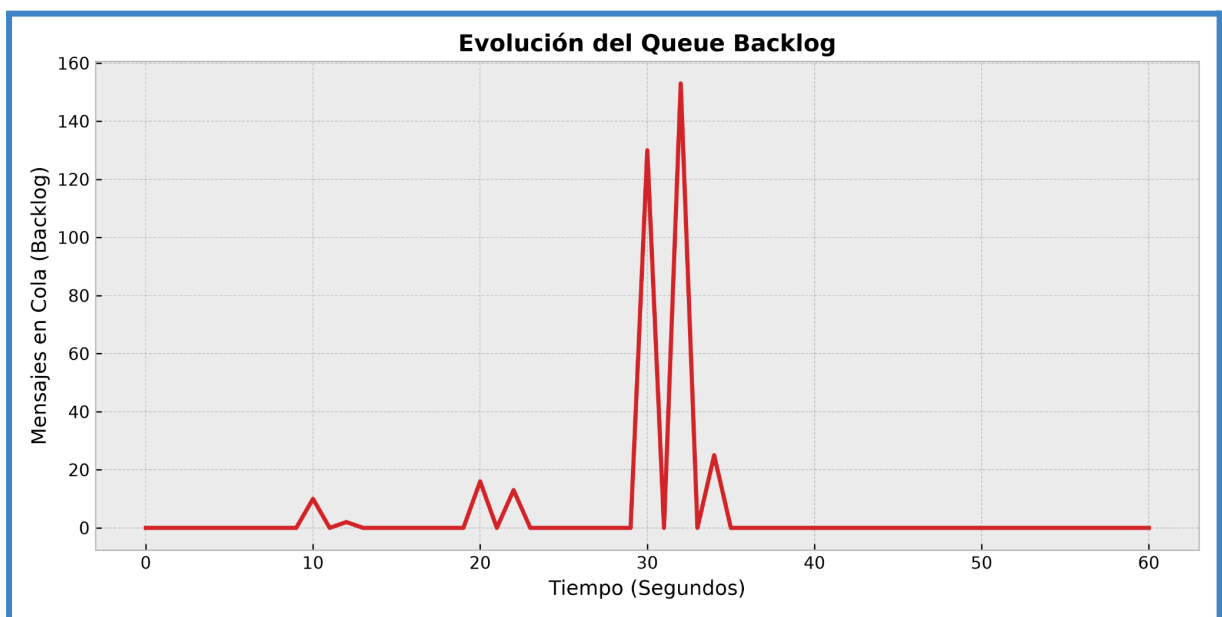


Imagen 4: Backlog en escenario Hotspot Numerado

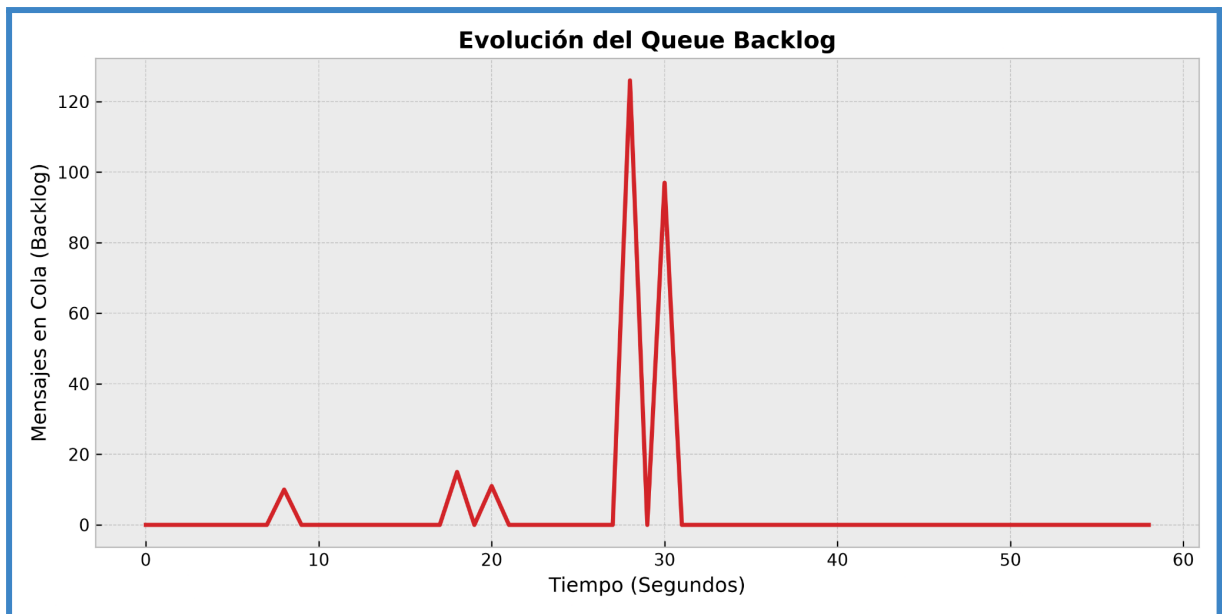


Imagen 5: Backlog en escenario Numerado

Estas gráficas evidencian la resiliencia asíncrona del sistema y su capacidad de suavizado de carga (load smoothing). Durante las fases iniciales de baja carga (Low Load Phase), el backlog se mantiene estable en niveles cercanos a cero. Sin embargo, al inyectar la fase de picos repentinos (Sudden spikes), la tasa de llegada de peticiones supera momentáneamente la capacidad de procesamiento de los workers activos, provocando que la cola acumule un exceso temporal que llega a superar los 140 mensajes.

En este punto, RabbitMQ actúa exitosamente como un amortiguador, evitando cualquier pérdida de mensajes. La gráfica demuestra que el sistema está perfectamente desacoplado, tras el pico, los workers logran drenar la cola por completo en cuestión de segundos, devolviendo el backlog a cero y restaurando el estado de reposo.

Rendimiento vs Trabajadores (Throughput vs workers)

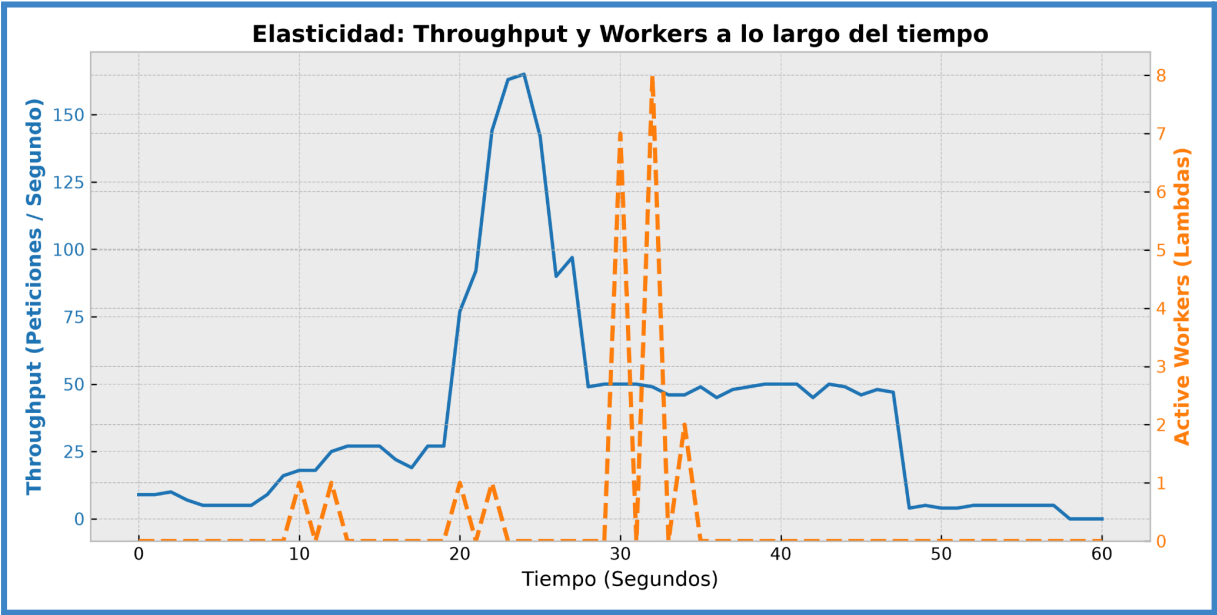


Imagen 6: Elasticidad en escenario No Numerado

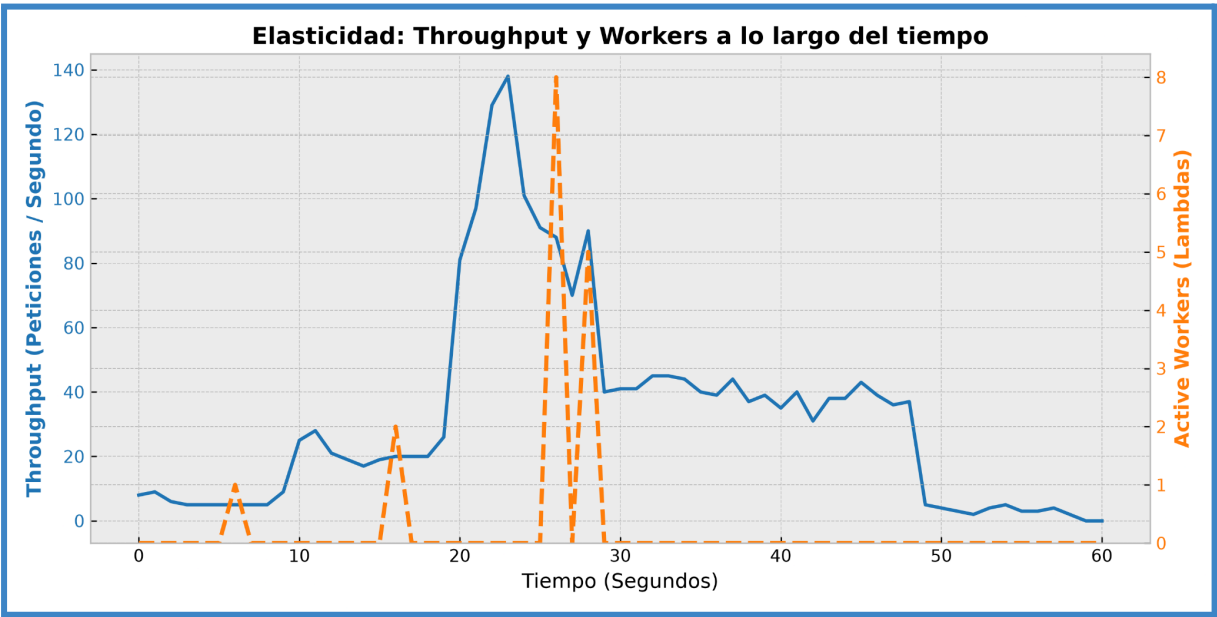


Imagen 7: Elasticidad en escenario Hotspot Numerado

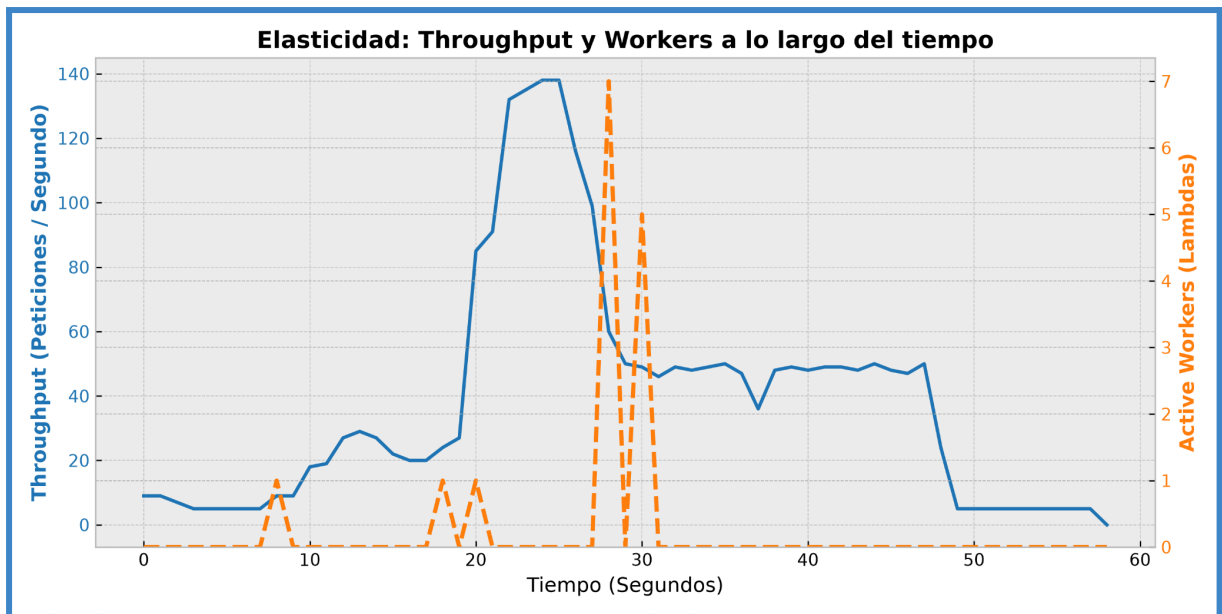


Imagen 8: Elasticidad en escenario Numerado

Esta visualización de doble eje valida la correcta implementación matemática del escalado dinámico. La línea punteada naranja (trabajadores AWS Lambda activos) persigue fielmente a la curva azul de peticiones procesadas por segundo (Throughput).

El autoescalador detecta el incremento de mensajes en el backlog y, aplicando la fórmula $N = \frac{B}{Tr \times C}$, inyecta dinámicamente hasta 8 workers concurrentes para absorber los picos de hasta 140 peticiones por segundo. Lo más destacable de estas gráficas es el cumplimiento del requisito de evitar el sobre-aprovisionamiento (avoid over-provisioning) en el instante exacto en que la cola de RabbitMQ se vacía o el tráfico disminuye a la fase de enfriamiento, el número de workers desciende de inmediato a cero, optimizando drásticamente el consumo de recursos de cómputo.

Percentiles de latencia (Latency percentiles)

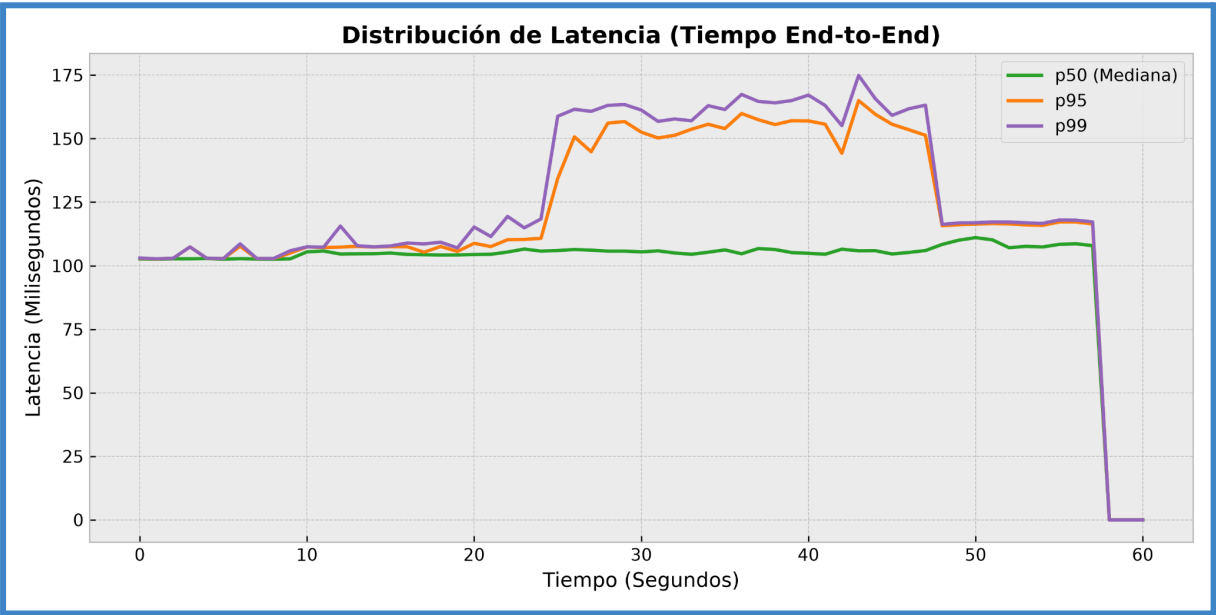


Imagen 9: Latencia en escenario No Numerado

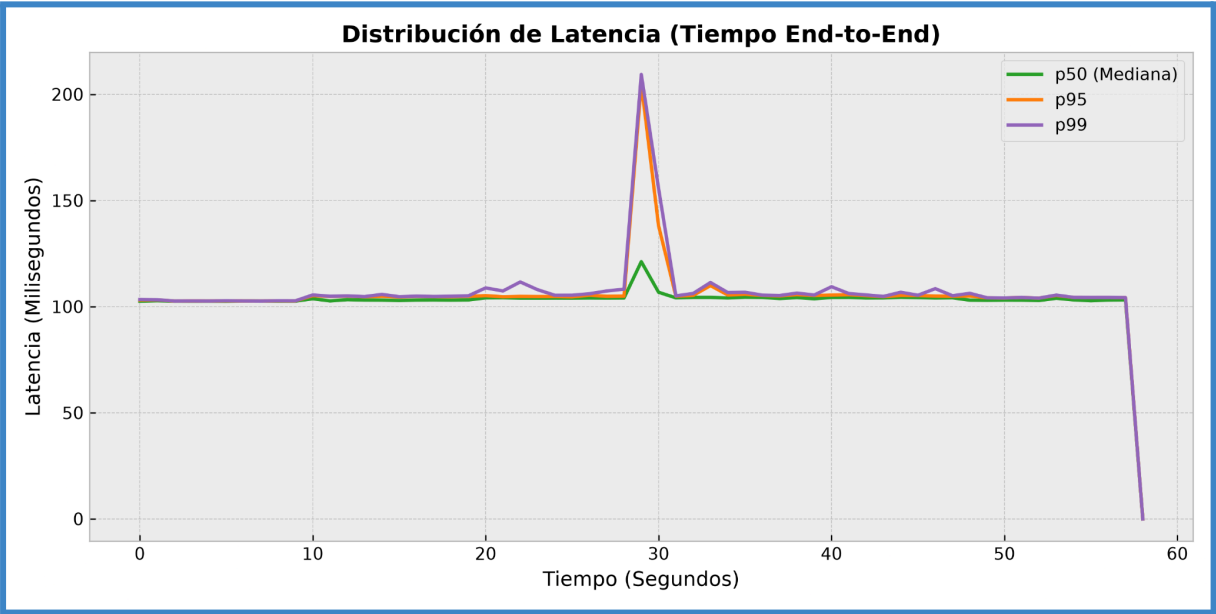


Imagen 10: Latencia en escenario Hotspot Numerado

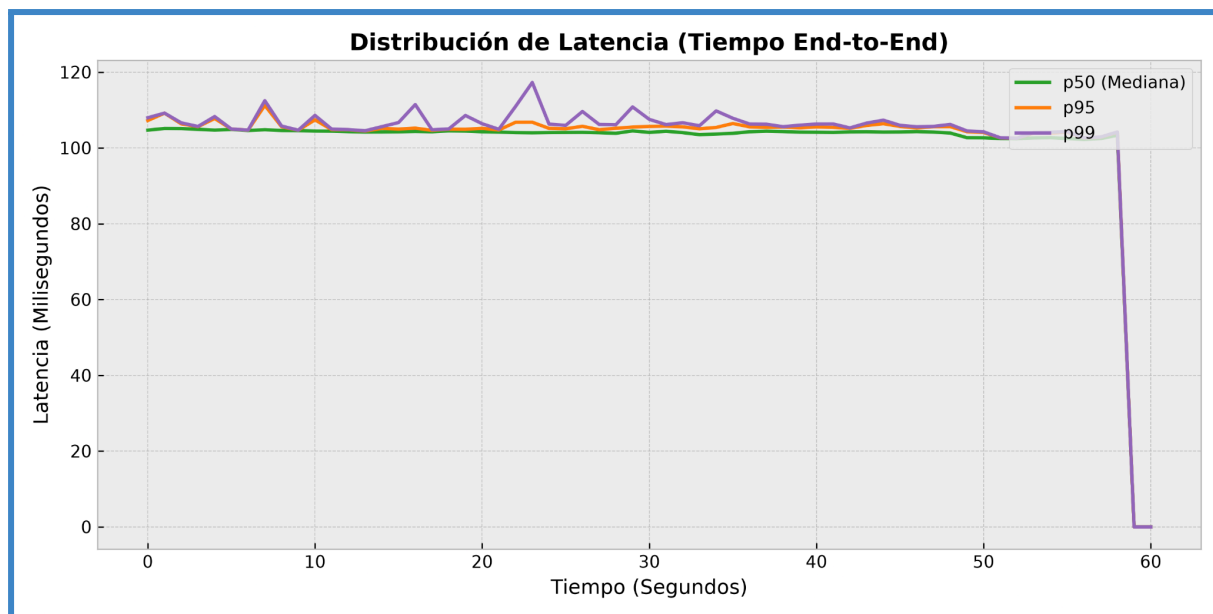


Imagen 11: Latencia en escenario Numerado

Las gráficas de latencia revelan el impacto físico y el trade-off directo de implementar un modelo de consistencia fuerte bajo diferentes niveles de estrés.

- **Línea base (Correctness garantizado):** En los tres escenarios evaluados, la latencia mediana (p50, línea verde) forma una línea recta casi perfecta en el umbral de los 103 a 105 milisegundos. Esto valida matemáticamente la ejecución del retraso artificial obligatorio de 100 ms (`time.sleep(0.100)`) exigido por el diseño, sumado a un mínimo sobrecoste de red y procesamiento de apenas 3 a 5 ms.
- **Escenario A - Entradas No Numeradas (Contención moderada):** En este caso, todas las funciones Lambda concurrentes ejecutan un comando `UPDATE` sobre la misma fila central (`event_id = 1`). Los bloqueos pesimistas a nivel de fila (Row-level locks) obligan a las peticiones secundarias a esperar su turno, lo que genera una penalización constante que eleva la latencia del p99 a un máximo de 174.85 ms durante el pico de tráfico.
- **Escenario B - Carga Hotspot (Alta contención):** Esta gráfica muestra un pico masivo donde el p99 se dispara hasta los 209.35 ms. Al forzar deliberadamente que el 80% del tráfico colisione intentando adquirir el 5% de los asientos VIP, se generan bloqueos severos en las sesiones de PostgreSQL. Las Lambdas quedan en estado de espera transaccional hasta que se libera la sentencia `SELECT ... FOR UPDATE` del asiento demandado.
- **Escenario C - Carga No Numerada:** En la gráfica plana, el percentil 99 (p99) apenas alcanza un pico de 117.27 ms. Al distribuir aleatoriamente las solicitudes de compra entre 100.000 asientos distintos, la probabilidad matemática de que dos workers intenten bloquear el mismo registro al mismo tiempo es casi nula. Sin colisiones de disco, la base de datos no encola peticiones y el sistema alcanza un rendimiento óptimo.

En conclusión, los picos de degradación en el p99 demuestran de forma fehaciente que el sistema prefiere penalizar la velocidad de respuesta en los extremos de alta demanda antes que permitir condiciones de carrera o corromper la integridad de los datos, logrando un 100% de exactitud y anulando matemáticamente cualquier posibilidad de sobreventa.

Análisis

Identificación de cuellos de botella (Bottlenecks identified)

El principal cuello de botella identificado en el sistema es la capa de base de datos relacional (PostgreSQL) bajo escenarios de alta contención. Si bien la capa de procesamiento computacional conformada por instancias de AWS Lambda permite un escalado horizontal elástico e independiente, la capacidad de confirmación de las transacciones queda estrictamente limitada por los bloqueos a nivel de disco de la base de datos.

Al aplicar un control de concurrencia mediante bloqueos pesimistas explícitos (`SELECT ... FOR UPDATE`), cualquier escenario que concentre múltiples peticiones sobre el mismo registro —como las entradas "No numeradas" o la demanda de los asientos VIP en el *Escenario Hotspot* provoca que las transacciones se sistematicen en colas de espera secuenciales en el motor de PostgreSQL. Por lo tanto, el throughput máximo se encuentra acotado por la capacidad de resolución de bloqueos del gestor de base de datos, independientemente de que se sigan añadiendo más workers sin estado.

Comportamiento del escalado (Scaling behavior explained)

El mecanismo de escalabilidad de este diseño se rige por una directriz dinámica fundamentada en el estado de la cola asíncrona (Backlog) de RabbitMQ. El componente Auto-scaler evalúa en tiempo real las fluctuaciones de carga $Z(t)$ y aplica matemáticamente la siguiente fórmula de aprovisionamiento:
$$N = \frac{B}{(Tr \times C)}$$

Donde:

- **B:** Backlog o número de mensajes esperando en la cola.
- **Tr:** Tiempo de respuesta objetivo del sistema, establecido en 2 segundos.
- **C:** Capacidad teórica por trabajador activo, fijada en 10 mensajes por ráfaga (basada en la configuración de prefetch).

El comportamiento derivado de esta configuración permite al sistema reaccionar a los picos masivos inyectando rápidamente workers (como se observó al alcanzar el tope de 140 req/sec) y, lo más crítico para la eficiencia de costes, desescalando la infraestructura a 0 workers en el instante exacto en que la cola se drena por completo. Esta elasticidad impecable satisface directamente la exigencia de evitar el sobre-aprovisionamiento (avoid over-provisioning).

Discusión de compensaciones (Trade-offs discussed)

La evaluación arquitectónica empírica visibiliza de forma nítida la compensación técnica clásica (trade-off) del desarrollo de sistemas distribuidos: el sacrificio del rendimiento de latencia en los percentiles extremos en favor de lograr la garantía absoluta de la consistencia de los datos.

Al requerir un sistema sin margen de error en el que la sobreventa de entradas esté estrictamente prohibida, se estableció un modelo de consistencia fuerte apoyado por los mencionados bloqueos pesimistas. La penalización se observa de manera evidente al contrastar las gráficas experimentales. Mientras que en entornos sin colisiones (Carga numerada) las transacciones incurren únicamente en el recargo de la latencia artificial exigida (rondando los 105 ms estables a nivel del p50), bajo los severos niveles de estrés direccional de un Hotspot, el bloqueo de las filas hace que la latencia en el percentil 99 (p99) se degrada considerablemente desde los ~105 ms base hasta los 209 ms.

A pesar de esta degradación comprobada en la velocidad de atención para el 1% de las peticiones más congestionadas, el diseño asume conscientemente este trade-off como el coste ineludible para garantizar un 100% de exactitud y corrección transaccional frente a escenarios críticos de venta concurrente.

Preguntas Conceptuales

Compromiso entre Consistencia y Escalabilidad

¿Qué modelo de consistencia se ha elegido y por qué? (What consistency model did you choose and why?)

Para el diseño de este sistema, se ha seleccionado un modelo de consistencia fuerte (Strong Consistency). La justificación principal radica en el cumplimiento estricto del requisito funcional que prohíbe de manera absoluta la sobreventa de entradas ("No overselling allowed"). Para garantizar esto, se ha implementado una estrategia de control de concurrencia mediante bloqueo pesimista (Pessimistic Locking) utilizando la instrucción SQL `SELECT ... FOR UPDATE` a nivel de fila en la base de datos PostgreSQL. Esta decisión asegura que, si múltiples instancias computacionales (Lambdas) intentan adquirir la misma entrada o alterar el mismo contador global en el mismo milisegundo, la base de datos obligue a las peticiones secundarias a esperar su turno de forma secuencial, haciendo matemáticamente imposible que se venda un ticket por duplicado.

¿Cómo cambiaría el comportamiento del sistema si se cambiara a un modelo más débil/fuerte? (How would your system behavior change if you switched to a weaker/stronger model?)

Si el sistema operara bajo un modelo de consistencia eventual (Eventual Consistency) por ejemplo, validando la disponibilidad de asientos mediante una caché en memoria sin implementar bloqueos transaccionales en disco, la escalabilidad mejoraría drásticamente y el rendimiento transaccional (throughput) alcanzaría volúmenes masivos. Al no existir cuellos de botella generados por la espera de bloqueos, las instancias podrían operar en paralelo de forma casi ilimitada. Sin embargo, el sistema perdería su garantía de exactitud (Correctness). Esto provocaría la venta recurrente de asientos duplicados durante los picos de alta concurrencia, lo que obligaría a implementar mecanismos de resolución de conflictos a posteriori, como la compensación manual a clientes afectados por la corrupción de inventario.

¿Cuál es el impacto en la corrección y el rendimiento? (What is the impact on correctness and performance?)

El impacto asume una compensación técnica (trade-off) directa y medible, el modelo garantiza un 100% de exactitud transaccional (Correctness garantizado), pero penaliza severamente el rendimiento máximo del sistema bajo estrés. La limitación principal es que el throughput queda acotado por la capacidad de la base de datos para gestionar y liberar los bloqueos en disco. Como se evidenció en la evaluación experimental, durante escenarios de alta contención (como el Hotspot), las esperas impuestas por la consistencia fuerte degradan la latencia en el percentil 99 (p99), elevándola de una base estable de ~105 ms a picos superiores a los 200 ms.

Compromisos entre Tolerancia a Fallos y Rendimiento

¿Cómo afectan los reintentos, la idempotencia y el manejo de fallos al rendimiento? (How do retries, idempotency, and failure handling affect performance?)

La implementación de mecanismos de resiliencia y tolerancia a fallos introduce un coste computacional y de latencia considerable. Para lograr la idempotencia requerida, cada worker (AWS Lambda) está obligado a ejecutar una consulta `SELECT` adicional a la base de datos de manera previa para verificar si el identificador único (`request_id`) ya se encuentra en la tabla de registros procesados. A esto se suma el manejo del ciclo de vida del mensaje en el middleware de mensajería (RabbitMQ), el cual exige el envío de confirmaciones positivas o negativas (`basic_ack` o `basic_nack`) a través de la red, lo que inevitablemente incrementa el tiempo total del flujo de ejecución.

¿Qué sobrecostos podría introducir la tolerancia a fallos? (What overheads could fault tolerance introduce?)

Los principales sobrecostos técnicos introducidos en la arquitectura son:

- **Aumento de la carga de Entrada/Salida (I/O) en disco:** Debido a las consultas repetitivas de verificación de estado y las escrituras necesarias para salvaguardar el progreso de las transacciones.
- **Mayor consumo de conexiones a la base de datos:** Al requerir operaciones extendidas por cada petición.
- **Latencia de red adicional:** Generada por el tránsito bidireccional continuo entre los workers sin estado, la base de datos persistente y la cola de mensajería para confirmar la manipulación segura de cada paquete.

¿Existe un punto en el que el aumento de la fiabilidad reduzca significativamente el rendimiento? (Is there a point where increased reliability significantly reduces throughput?)

Sí. Para poder garantizar la semántica de entrega "*al menos una vez*" (*At-least-once semantics*) y asegurar que existan cero pérdidas de compras, el sistema sacrifica directamente su throughput puro (rendimiento bruto). En entornos de producción sometidos a ráfagas intensivas de tráfico, la sumatoria de estas comprobaciones individuales, reintentos encolados y validaciones de redundancia crea un techo de procesamiento máximo. En este punto, se asume la disminución de la velocidad extrema en favor de dotar al sistema de una robustez y fiabilidad inflexibles.

Uso de la IA

Durante la elaboración y desarrollo de este proyecto, se han empleado herramientas y asistentes basados en Inteligencia Artificial (LLM) bajo un enfoque estrictamente guiado e instrumental. Su utilización se ha delimitado de forma transparente a las siguientes áreas operativas:

- **Aceleración de desarrollo:** La IA se ha utilizado para agilizar la generación de estructuras base y la redacción de scripts repetitivos tanto en Python como en sentencias SQL.
- **Visualización de datos:** Se ha empleado la asistencia de la IA para generar el código automatizado de las gráficas de validación mediante la librería Matplotlib, extrayendo y procesando la información directamente desde los archivos CSV generados por las pruebas.
- **Soporte arquitectónico:** Las herramientas sirvieron como apoyo consultivo para discutir y contrastar diferentes enfoques arquitectónicos respecto a la implementación y el comportamiento de los servicios administrados en la nube de AWS.